

Similarly, we perform the same computation on the right constraint:

$$LHS = \int q(x_0, x_1) dx_0 = \int \int q(x_0, x_1 | S) q(S) dS dx_0 \quad (18)$$

$$= \int q(S) \int q(x_0, x_1 | S) dx_0 dS \quad \text{change the order of integration} \quad (19)$$

$$= \int q(S) \int \prod_i^N \hat{q}(x_0^i, x_1^i | S) d(x_0^1, \dots, x_0^N) dS \quad \text{independent assumption of each row in training pair} \quad (20)$$

$$= \int q(S) \prod_i^N \int \hat{q}(x_0^i, x_1^i | S) dx_0^i dS \quad \text{integrals of independent products} \quad (21)$$

$$= \int q(S) \prod_i^M \sum_j \hat{q}(x_0^j, x_1^j | S) dS \quad \text{restricting to discrete values in supersets} \quad (22)$$

$$= \int q(S) \prod_i \hat{q}_1(x_1^i | S) dS \quad \text{Proposition 2} \quad (23)$$

$$= \int q(S) q_1(x_1 | S) dS = q_1(x_1) \quad \text{independent assumption of each row in point cloud} \quad (24)$$

$$(25)$$

A.1.3 PROOF OF HYBRID COUPLING

In the last, we would like to show even with our hybrid coupling, the marginal still fulfills the requirements. In particular, we define a new noises x'_0 after perturbation:

$$x'_0 = \sqrt{1 - \beta} x_0 + \sqrt{\beta} \epsilon, \epsilon \sim N(\epsilon; 0, I), \quad (26)$$

where $\beta \in [0, 1]$ is the blending coefficient. We denoted this as a conditional distribution $q(x'_0 | x_0)$, which has a form of $N(x'_0; \sqrt{1 - \beta} x_0, \beta)$. It is noted that since $\epsilon \sim N(\epsilon; 0, I)$, each row of x'_0 is also IID given x_0 , i.e., $q_0(x'_0 | x_0) = \prod_i^N \hat{q}_0(x'_0^i | x_0^i)$. Due to the independent properties, it is sufficient to show that:

$$\int q(x'_0, x_1 | S) dx'_0 = q_1(x_1 | S), \int q(x'_0, x_1 | S) dx_1 = q_0(x'_0). \quad (27)$$

For the sake of simplicity, we remove all the index i and shape S in the folloings. We first show the left constraint:

$$\int q(x'_0, x_1) dx'_0 = \int \int q_0(x'_0, x_0, x_1) dx_0 dx'_0 \quad (28)$$

$$= \int \int q_0(x'_0 | x_0) q(x_0, x_1) dx_0 dx'_0 \quad (29)$$

$$= \int q(x_0, x_1) \int q_0(x'_0 | x_0) dx'_0 dx_0 \quad (30)$$

$$= \int q(x_0, x_1) (1) dx_0 \quad (31)$$

$$= q(x_1) \quad (32)$$

By Proposition 1, we can show $q(x_1)$ still converge to the right CDF if M is sufficient large.

On the other hand, we show that:

$$\int q(x'_0, x_1) dx_1 = \int \int q_0(x'_0, x_0, x_1) dx_0 dx_1 \quad (33)$$

$$= \int \int q_0(x'_0, x_0) dx_0 \quad (34)$$

$$= \int \int q_0(x'_0|x_0) q(x_0) dx_0 \quad (35)$$

$$= N(0, I) \quad (36)$$

where the last equality is obtained by inserting $q(x_0) = N(0, I)$ and $q_0(x'_0|x_0) = N(x'_0; \sqrt{1-\beta}x_0, \beta I)$.

B IMPLEMENTATION DETAILS

Training Details. We implement our networks using PyTorch (Paszke et al., 2019) and run all experiments on a GPU cluster with four A100 GPUs. We employ the Adam optimizer (Kingma, 2014) to train our model with a learning rate of 2×10^{-4} and an exponential decay of 0.998 every 1,000 iterations. Following LION (Zeng et al., 2022), we use an exponential moving average (EMA) of our model with a decay of 0.9999. Specifically, we train our unconditional generative model for approximately 600,000 iterations with a batch size of 256, taking about four days to complete. It is noted that we use larger batch sizes and more training iterations compared to existing work, including (Zhou et al., 2021; Wu et al., 2023), to effectively compare various training paradigms (Figures 6 & 7). This choice ensures our training procedure has higher stability and converges properly. Additionally, we want to highlight that the online subsampling procedure introduces negligible overhead in the training process (merely requiring additional indexing of a cached array).

Network Architecture. For the network architecture, we adopt the same structure as PVD (Zhou et al., 2021) and employ PVCNN (Liu et al., 2019) as our vector field network for the unconditional generation task. In the shape completion task, we use an additional 256-dimensional latent vector to represent the input partial point cloud, which is then injected into PVCNN. To do so, we use another PVCNN follow LION (Zeng et al., 2022) to extract the latent vector from the partial point cloud.

Generated Shape Normalization. To ensure a fair evaluation among different baselines in the unconditional task, we convert the inference results of various generative methods into a common coordinate domain. For all baseline methods, including PVD (Zhou et al., 2021), DiT-3D (Mo et al., 2023), LION (Zeng et al., 2022), and PSF (Wu et al., 2023), we respect the original normalization adopted in their training procedures. Since all these methods compute a global mean coordinate and global scale ratio across all training shapes, we use these two quantities to reverse the normalization on the generated shape, based on the values obtained from the training set. This procedure aligns with existing baselines, such as (Yang et al., 2019; Zeng et al., 2022).

C APPROXIMATED OT

C.1 IMPLEMENTATION DETAILS

In the following, we outline the procedure for the approximated Optimal Transport (OT) employed for supersets with size $M > 10,000$. We are motivated to use an approximated OT because the computational time for the exact method, *i.e.*, the Hungarian algorithm (Kuhn, 1955), is prohibitively large for a large superset size. Given supersets of size 10,000, it already takes 220 seconds. Considering the complexity of $O(M^3)$, this is unlikely to be scalable for larger set sizes, *e.g.*, $M = 100,000$, for the thousands of training shapes in each ShapeNet category, even if we move this computation offline. Therefore, we resort to an approximation procedure that utilizes Wasserstein gradient flow to obtain our point superset X_1 by progressively transporting a noise X_0 .

OT Distances. To enable an optimization procedure to move the points, we need to define an objective to be optimized, which is the 2-Wasserstein distance in our case. In particular, the OT distance can be defined as:

$$W(q_0, q_1) := \min_{q \in \Gamma(q_0, q_1)} \int C(x_0, x_1) q(x_0, x_1) dx_0 dx_1, \quad (37)$$

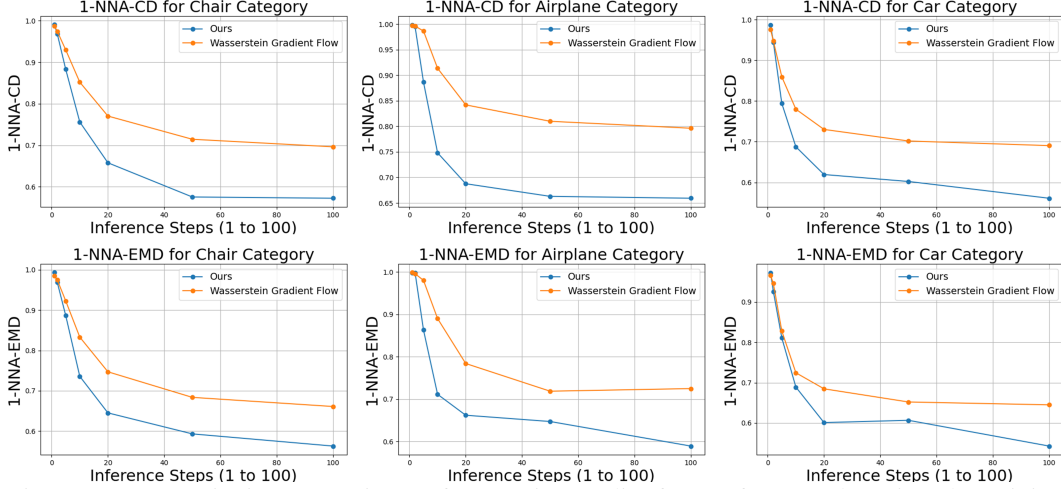


Figure 11: Quantitative comparisons of generation quality for our framework against the training pairs obtained by 1-step Wasserstein Gradient Flow. We also show 1-NNA-CD (top) and 1-NNA-EMD (bottom) for Chair (left), Airplane (middle), and Car (right). Note that a value closer to 50% indicates better performance.

where $\Gamma(q_0, q_1)$ represents all couplings with marginals q_0 and q_1 . However, explicitly computing the OT is not tractable, so a relaxation of this cost is introduced by adding an entropic barrier term weighted by ϵ for solving various problem, including (Léonard, 2012). This relaxation enables an efficient solution by employing the Sinkhorn algorithm, which can be highly parallelized on the GPU. As shown in (Feydy et al., 2019), this entropic barrier introduces bias in measuring the distance. Even when the marginals in the above equation are equal, i.e., $q_0 = q_1$, the distance might not be zero, leading to poor gradients. To address this issue, Genevay et al. (2018) introduce correction terms to the objective, forming the Sinkhorn divergence. Additionally, Feydy et al. (2019) provide an efficient, memory-saving GPU implementation that scales to millions of samples.

Optimization Procedure. With the defined objective, our optimization will compute the gradient with respect to this objective to update the current point samples. Initially, we initialize the coordinates of a set of points as the noise superset x_0 . At each optimization iteration, we compute the Sinkhorn divergence between the current point set and the target superset X'_1 . By repeatedly minimizing the OT distance in the form of Sinkhorn divergence, we obtain a deformed point set that fits the given superset well. Finally, we use this deformed point set as our point superset to represent the shape S .

Note that using this computed superset as X_1 introduces approximation errors compared to true samples X'_1 . However, we empirically observe that the procedure usually converges well with only small errors, and in our experiments, the large superset size typically outweighs the introduced error (as shown in Section 4.3).

C.2 TRAINING PAIRS BY 1-STEP APPROXIMATION

We also compare our approach with training pairs obtained through the optimization procedure described above. In this experiment, we perform only a single optimization iteration on point clouds and noises of size 2,048 and set β to 0.0. For each training iteration, we use the optimized results, X_0 and X_1 , as our training pair. The quantitative comparison results are shown in Figure 11. Our approach consistently outperforms the 1-step Wasserstein Gradient Flow across all categories and most inference timesteps.

D ADDITIONAL QUANTITATIVE COMPARISON

D.1 COVERAGE METRIC (COV)

Following Zhou et al. (2021); Zeng et al. (2022), we also evaluate coverage (COV), a metric that measures the diversity of generated 3D shapes. COV calculates the proportion of testing shapes that